Batch: B2	Roll No.: 16010124107
Experiment No. 8	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

TITLE: Implementation of Deadlock Avoidance Policy-Banker's Algorithm

---

UUAIM: Implementation of Deadlock Avoidance Policy-Banker's Algorithm

---

Expected Outcome of Experiment:

CO3 Describe the problems related to process concurrency and the different synchronization mechanisms available to solve them.

Books/ Journals/ Websites referred:

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**

2. **Achyut S. Godbole , Atul Kahate "Operating Systems" McGraw Hill Third Edition.**

3. **William Stallings, "Operating System Internal & Design Principles", Pearson.**

4. **Andrew S. Tanenbaum, "Modern Operating System", Prentice Hall.**

Pre Lab/ Prior Concepts:

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.

It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.

By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order.

Components of the Banker's Algorithm

The following Data structures are used to implement the Banker's Algorithm:

Let 'n' be the number of processes in the system and 'm' be the number of resource types.

1. Available

It is a 1-D array of size 'm' indicating the number of available resources of each type.

Available[j] = k means there are 'k' instances of resource type Rj

2. Max

It is a 2-d array of size 'n*m' that defines the maximum demand of each process in a system.

Max[i, j] = k means process Pi may request at most 'k' instances of resource type Rj.

3. Allocation

It is a 2-d array of size 'n*m' that defines the number of resources of each type currently allocated to each process.

Allocation[i, j] = k means process Pi is currently allocated 'k' instances of resource type Rj

4. Need

It is a 2-d array of size 'n*m' that indicates the remaining resource need of each process.

Need [i, j] = k means process Pi currently needs 'k' instances of resource type Rj

Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

Key Concepts in Banker's Algorithm

Safe State: There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.

Unsafe State: Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

Implementation details:

```
#include <bits/stdc++.h>
using namespace std;

void print(vector<bool> &v){
    for(int i:v) cout << i << " ";
    cout << endl;
}
void print(vector<int> &v){
    for(int i:v) cout << i << " ";
    cout << endl;
}

int main() {
    int n,m;
    cout << "Enter number of processes and resources\n";
    cin >> n >> m;
    cout << "Enter available vector\n";
    vector<int>available(m);
    for(int &i: available){
        cin >> i;
    }
    vector<int>work(available.begin(),available.end());
    vector<bool>finish(n,false);
    cout << "Enter max matrix\n";
    vector<vector<int>>matrix(n, vector<int>(m));
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cin >> matrix[i][j];
```

```
        }
    }
    cout << "Enter allocation matrix\n";
    vector<vector<int>>allocation(n, vector<int>(m));
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cin >> allocation[i][j];
        }
    }
    vector<vector<int>>need(n, vector<int>(m));
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            need[i][j] = matrix[i][j] - allocation[i][j];
        }
    }
    int ind=0;
    vector<int>ans(n);
    while(true){
        bool found = false;

        for(int i=0;i<n;i++){
            if(!finish[i]){
                bool flag = true;

                for(int j=0;j<m;j++){
                    if(need[i][j] > work[j]){
                        flag = false;
                        break;
                    }
                }

                if(flag){
                    ans[ind++]=i;
                    for(int j=0;j<m;j++){
                        work[j] += allocation[i][j];
                    }
                    finish[i] = true;
                    found = true;
```

```
                }
            }
        }
        if(!found) break;
    }
    print(finish);
    print(ans);
}
```

Output:

```
Enter number of processes and resources
5 3
3 3 2
Enter max matrix
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter allocation matrix
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
1 1 1 1 1
1 3 4 0 2
PS C:\Users\syeda\OneDrive\Desktop\semiv>
```

The safe order of processes is as follows: P1->P3->P4->P0->P2

Conclusion :

The Banker's Algorithm determines whether a system is in a safe state by checking if all processes can complete without deadlock. It computes the Need matrix as Max minus Allocation and compares each process's needs with available resources. If a process can execute, it releases its resources after completion and is marked finished. This process repeats until all processes finish or no further allocation is possible. If all processes complete, the system is safe; otherwise, it is unsafe.

Post Lab Descriptive Questions

A. Assume that there are 5 processes, P0 through P4, and 4 types of resources. At T0 we have the following system state:

Max Instances of Resource Type A = 3

Max Instances of Resource Type B = 17

Max Instances of Resource Type C = 16

Max Instances of Resource Type D = 12

	Allocation Matrix (N0 of the allocated resources By a process)				**Max Matrix Max resources that may be used by a process**			
	A	**B**	**C**	**D**	**A**	**B**	**C**	**D**
P_0	0	1	1	0	0	2	1	0
P_1	1	2	3	1	1	6	5	2
P_2	1	3	6	5	2	3	6	6
P_3	0	6	3	2	0	6	5	2
P_4	0	0	1	4	0	6	5	6

Find whether the system is in a safe state or not with the process sequence.

First, the available resources are calculated by subtracting total allocated resources from the maximum instances, giving Available = (1, 5, 2, 0).

The Need matrix is computed as Max − Allocation.

Checking processes sequentially, P0 can execute since its needs are within available resources, and after completion, it releases resources, increasing availability. Next, P3 can execute, followed by P1, then P2, and finally P4, as at each step the available resources are sufficient to satisfy the remaining needs.

Since all processes can complete successfully in the sequence P0 → P3 → P1 → P2 → P4, the system is safe and no deadlock will occur.

Verification:
The system is in a safe state with the process sequence per our code.

```
Enter number of processes and resources
5 4
Enter available vector
1 5 2 0
Enter max matrix
0 2 1 0
1 6 5 2
2 3 6 6
0 6 5 2
0 6 5 6
Enter allocation matrix
0 1 1 0
1 2 3 1
1 3 6 5
0 6 3 2
0 0 1 4
1 1 1 1 1
0 3 4 1 2
PS C:\Users\sveda\OneDrive\Desktop\semiv>
```